

Using Modules in a Real Project

EREZ STRAUSS

Using Modules in a Real Project



C++26 · CMake · Clang · GCC

Erez Strauss · CppCon 2026

github.com/erez-strauss/cxx-modules

C++ Modules are usable, though integrating them takes work.



Agenda

1. **What we already know** - `#include` is paste; the header build is embarrassingly parallel
2. **Module structure** - PMIU, partitions, implementation units, BMI
3. **Build your project** - CMakeLists.txt, freeze before `project()`, `FILE_SET`, Ninja, `import std`, week-one traps
4. **Packaging** — mixed trees, and what goes in the tarball

By the end of this talk, you will be equipped to:

1. Describe the four module unit kinds,
2. Work with cmake to build modules based libraries and applications,
3. Plan an incremental adoption.



The examples library

'tally' is a ledger, Money, accounts, a journal.

Week-one walls: templates, out-of-line members, a header-only dep, a macro, a tree that cannot stop the world.

```
examples/  
  00-simple-example      # two files; g++ by hand vs cmake --preset  
  01-headers             # the library you already have (#include)  
  02-module              # PMIU, partitions, impl units, import std  
  03-macros-templates    # macros do not export; template reachability  
  04-mixed               # GMF wrap + dual-mode umbrella  
  05-timing              # include vs import std (measure; not a recipe)  
  06-trading             # OMS + post-trade, .a / .so, CPack
```



'import' is not paste

#include	import
Textual inclusion	Semantic import of a BMI
Named by file (<code>"tally/order.hpp"</code>)	Named by module (<code>tally</code>); BMI path comes from a module map
Macros travel	Macros do not travel
Order and include-guards matter	The BMI is already parsed
ODR is a social contract	The compiler can see across TUs
Embarrassingly parallel	Interface BMIs are a build-order edge

The left column is pre-modules, familiar: include guards, templates-in-headers, PCH. All the reason C++ builds hurt.

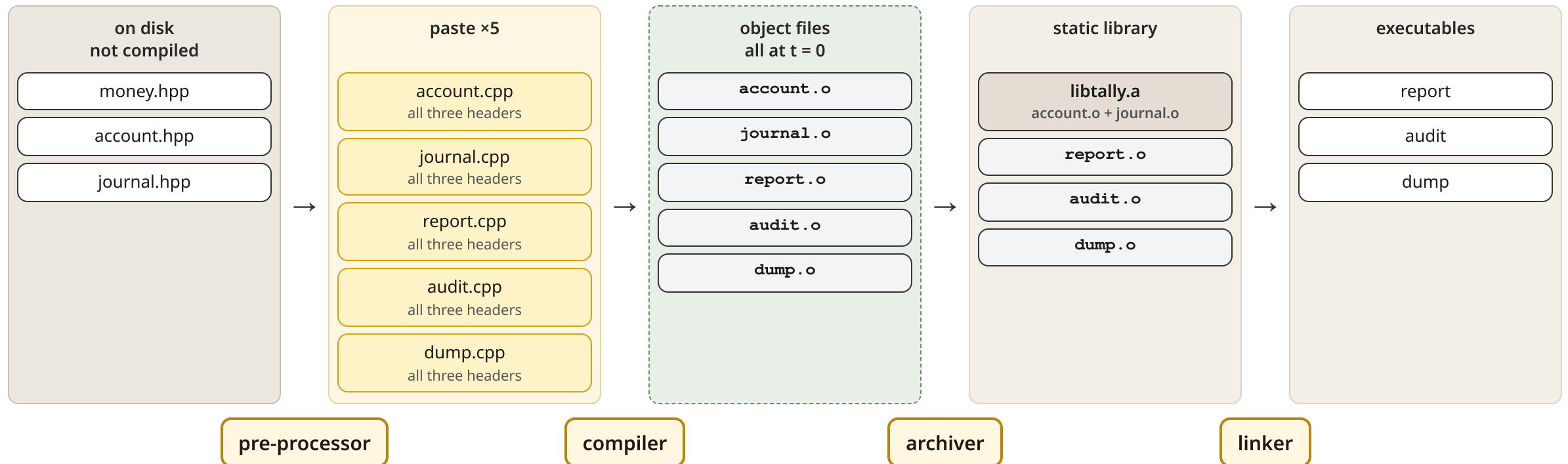
On the right: the new C++ modules way.

BMI = Built Module Interface (Clang `.pcm`, GCC `.gcm`).

The rest of the talk is that table, made concrete.



Header Project - build



Modules Vocabulary

A **named module** is a set of *module units* with the same name.

Unit	Declaration	Produces BMI?
Primary module interface unit	<code>export module tally;</code>	yes — this is what clients <code>import</code>
Interface partition	<code>export module tally:money;</code>	yes — only visible <i>inside</i> <code>tally</code>
Internal partition	<code>module tally:detail;</code>	no — module partition implementation
Implementation unit	<code>module tally;</code>	no — object file only

GMF = global module fragment (`module;` ... then the module declaration).



One primary interface. Exactly one.

```
export module tally;           // the PMIU – clients write: import tally;

export module tally:money;     // a partition, not a second module
module tally;                  // implementation; implicitly imports the PMIU
export module tally.money;     // a *different* named module. Dots are not hierarchy.
```

If you remember one rule: **clients import a name, not a file.**
Partitions are an implementation detail of that name.



Anatomy of a module unit

```
module;                                // optional global module fragment
#include "third_party.hpp"              // only preprocessor directives here
                                        // not for the standard library – import std;

export module tally;                    // the module declaration

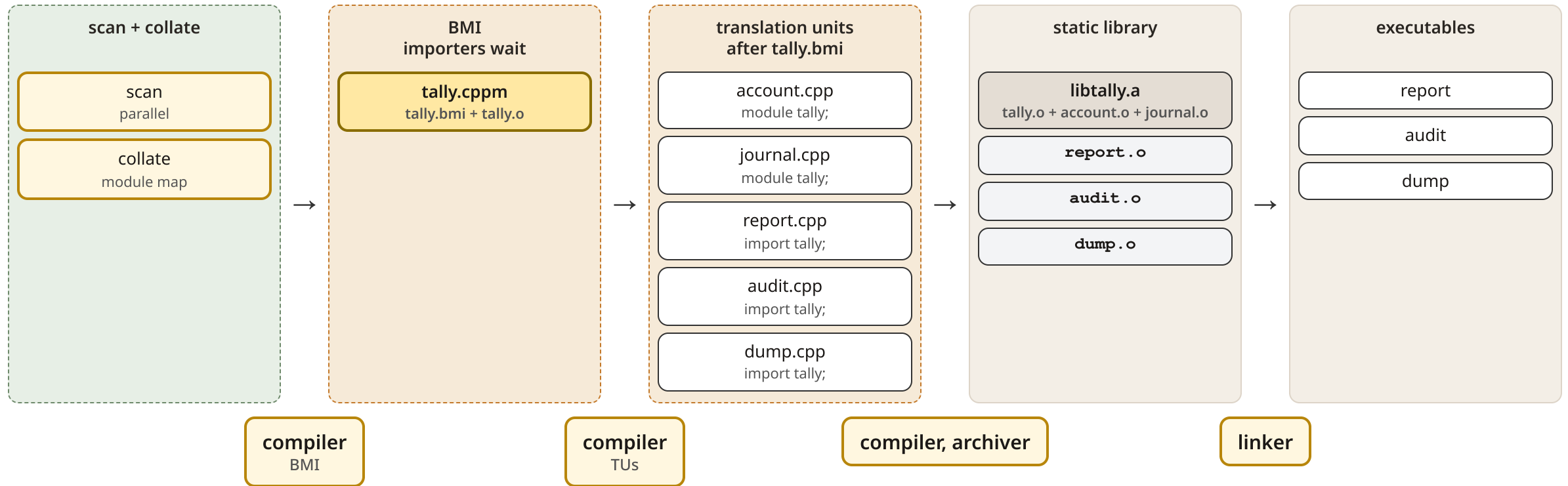
import std;                            // all imports, then other declarations
export import :money;

export namespace tally {               // only exported names are visible to importers
    class Account;
}
```

`#include` after `export module` makes those declarations **part of the module**.
Don't. Third-party headers go in the GMF. `std` is `import std;` (C++23+).



Modules based Project - build



examples/00-simple-example — you are the collator

Two files. Four commands. The module diagram, typed by hand.

```
g++ -std=c++26 -fmodules -c bits/std.cc -o std.o           # std BMI
g++ -std=c++26 -fmodules -c mym1.cppm -o mym1.o           # mym1 BMI + .o
g++ -std=c++26 -fmodules -c myu1.cpp -o myu1.o            # consumer: needs both BMIs
g++ -std=c++26 -fmodules -o myu1 myu1.o mym1.o std.o      # link objects, not BMIs
```

```
// mym1.cppm  -- my Module 1
export module mym1;
import std;
export const std::string& my_name();

// myu1.cpp  -- module User 1
import mym1;    // one import per line.
import std;
int main() { std::println("{} ", my_name()); }
```

The compile commands order is required. Reverse any of the first three compile lines and the compiler names the missing BMI.

In the example: `./compile-cmds.gcc.sh` .



Same graph, CMake is the collator

```
# UUID + CMAKE_CXX_STANDARD + CMAKE_CXX_EXTENSIONS OFF -- all before project()
add_executable(myu1 myu1.cpp)
target_sources(myu1 PRIVATE FILE_SET CXX_MODULES FILES mym1.cppm)
```

```
cmake --preset gcc-libstdcxx -Wno-dev
cmake --build build/gcc-libstdcxx -v    # compare this log to compile-cmds.gcc.sh
```

CMake scans, writes the module map, builds `std` and `mym1` BMIs, then `myu1`.
This target holds the module **and** `main`. A library uses `add_library` (next).

Presets: `gcc-libstdcxx` · `clang-libstdcxx` · `clang-libcxx`.

The `.pcm` / `.gcm` is a compiler input. The object files `.o` still have to be linked.
That is why the manual link line lists `std.o`.



Example 01 — a header based project

```
// include/tally/account.hpp
#pragma once
#include "tally/money.hpp"
class Account {
    void credit(Money<>);
    // ...
};
```

```
add_library(tally src/account.cpp src/journal.cpp)
target_include_directories(tally PUBLIC include)
```

Every consumer of `account.hpp` re-parses `money.hpp` and whatever *it* includes.



Example 02 — the same API, with a named module

```
export module tally;
import std;

export namespace tally {
    template<class Rep = long long> class Money { /* ... */ };
    class Account { /* ... */ };
    class Journal { /* ... */ };
}
```

```
import std;
import tally;

int main() {
    tally::Account cash{"cash"};
    std::println("{} ", cash.id());
}
```

This is the **whole application**. One `import` per line; `std` then `tally`.

On disk, `02-module` is already split (partitions + impl units). **Write one PMIU first.**



The CMake that makes this a project

```
cmake_minimum_required(VERSION 4.0)           # 4.0+ on purpose; older CMake, older problems
set(CMAKE_EXPERIMENTAL_CXX_IMPORT_STD "...")  # before project()
set(CMAKE_CXX_STANDARD 26)
set(CMAKE_CXX_STANDARD_REQUIRED ON)           # do not silently fall back
set(CMAKE_CXX_EXTENSIONS OFF)                # else Clang: gnu++ BMI vs c++ TUs
set(CMAKE_CXX_MODULE_STD ON)
project(tally LANGUAGES CXX)
add_library(tally)
target_sources(tally
    PUBLIC FILE_SET CXX_MODULES FILES tally.cppm
    PRIVATE src/account.cpp)                  # impl unit: not in the file set
```

`FILE_SET CXX_MODULES` is the named-module trick.

The UUID is `cmake experimental, standard, and extensions` — all before `project()`. Clang checks; GCC trusts.

Ninja 1.11+. Not Make. Not "list the `.cppm` as a normal source and hope."



What CMake actually does

Compiling is no longer embarrassingly parallel, code is only compiled once.

1. **Scan** each C++20 source for `import` / `export module` (P1689)
2. **Collate** those scans into a dependency graph
3. Build **BMI**s before the TUs that import them
4. Hand the compiler a **module map**: *this `import tally;` is that file*

The scan stays cheap because a module *name* can no longer be a macro (P3034, C++26); `import LIB;` may still expand.

You see this as extra ninja steps and `*.ddi` files.

That is the "real diagnostics" in the abstract: a missing BMI is a **named error**, not a mysterious undeclared identifier three includes deep.

The module diagram is this pipeline. Scan is parallel. **BMI production is the serial prefix.**



What C++26 changed for modules

Two small rules. Both already in the compilers on this laptop.

```
#define LIB tally
export module LIB;           // P3034R1 – ill-formed in C++26
import LIB;                  // still fine: imports may expand macros
```

The module *name* is now findable without running the preprocessor.
That is why the scan is as cheap as it is.

```
module tally;                // implementation unit
extern "C++" int main() {    // P3618R0 – well-formed in C++26
    // this is how you test what you did not export
}
```

`import std;` is C++23. Named modules are C++20.

C++26 is where the *build* rules got tightened.



Toolchain, September 2026

	Named modules	<code>import std;</code>
CMake	3.28, stable	3.30+, still behind <code>CMAKE_EXPERIMENTAL_CXX_IMPORT_STD</code>
Ninja	1.11	required (VS generator cannot consume <code>import std</code> BMIs)
Clang	16+	18.1.2+ · libc++ or libstdc++
GCC	14+ · still needs <code>-fmodules</code> (CMake adds <code>-fmodules-ts</code>)	15+
Header units	in the standard	not implemented in CMake

GCC 16 still says `sorry, unimplemented: private module fragment`.

Clang 22.1 makes **Reduced BMI** the default — a BMI now deliberately omits what importers cannot see.



Implementation units — examples/02-module

```
// tally.cppm
export module tally;
export class Account {
    void credit(Money<> amount); // declaration only
};
```

```
// src/account.cpp
module tally; // implicitly imports the PMIU
import std;

void tally::Account::credit(Money<> amount) {
    balance_ += amount;
}
```

```
add_library(tally)
target_sources(tally
    PUBLIC
        FILE_SET CXX_MODULES_FILES
            tally.cppm # produces a BMI
    PRIVATE
        src/account.cpp # produces an object file
        src/journal.cpp)
```

The implementation unit is a **regular source**. Do not put it in **FILE_SET** **CXX_MODULES**. Do not name it **.cppm**.



Partitions — examples/02-module

```
export module tally:money;      // interface partition
export template<class Rep> class Money { /* ... */ };

export module tally:account;    // module interface partition unit
export import :money;           // partitions import each other by short name
export class Account { /* ... */ };

export module tally;            // PMIU: this is the public face
export import :money;
export import :account;
export import :journal;
```

Clients still write `import tally;` .

They cannot write `import tally:money;` . That name is not for them.

The PMIU is a **table of contents**. Only implementation units `import :detail` — if the primary imports a PRIVATE partition, CMake errors at scan.

They are not submodules. Split when the PMIU is large, or two interface types need each other (cycles *inside* a module are OK). Do not invent a partition per header on day one.



The std module itself

Named modules arrived in C++20.

`import std;` arrived in C++23.

```
import std;           // namespace std, including C library names in std::
import std.compat;    // also ::printf, ::size_t, ... in the global namespace
import tally;         // example module
```

Do not `export import std;` from a leaf library.

Your consumers write `import std;` when they need it.

`std` is one BMI for the whole standard library.



The CMake gate for import std;

Must be set before `project()`:

```
cmake_minimum_required(VERSION 4.0)
set(CMAKE_EXPERIMENTAL_CXX_IMPORT_STD "...") # per version: Help/dev/experimental.rst
set(CMAKE_CXX_STANDARD 26)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF) # before project() - Clang gnu++ vs c++ is a hard error
set(CMAKE_CXX_MODULE_STD ON)
project(tally LANGUAGES CXX)
```

The UUID changes between CMake versions.

This repo's `cmake/EnableImportStd.cmake` picks the right one.

CMake 4.3 still warns: experimental. It works.

`CXX_MODULE_STD` is **per target**, and CMake will not add `import std` to a `cxx_std_23` target without being asked. A policy to flip that default is expected once the feature sees wider use — that is the shape the promise will take.



Compile time — what actually changes

`#include <vector>` in 200 TUs: parse vector 200 times.

`import std;` in 200 TUs: build the `std` BMI **once**, then load it.

Three numbers to measure, not one:

1. **Cold build** — first `std` BMI is several seconds. Can lose.
2. **Rebuild of many TUs** — this is where you win.
3. **Incremental edit of one TU** — usually a win after the BMI exists.

Example 05 is 16 or 64 TUs × heavy headers vs `import std; .`

Run `scripts/time-builds.sh` . Quote *your* machine — the next slide is only mine.



One machine, one run

i9-13900H · 20 threads · GCC 16.2.1 · Clang 22.1.8 · CMake 4.3 / Ninja 1.13 · Release
· **ccache off**

Rebuild of all TUs	g++ 16	g++ 64	clang 16	clang 64
headers	1.62 s	5.99 s	2.08 s	8.02 s
<code>import std;</code>	0.53 s	1.95 s	0.13 s	0.41 s
	3.1×	3.1×	16×	20×

Per TU at 64: a header TU **2.3 – 3.5 s**, an `import std;` TU **0.13 – 0.92 s**.

Both sides scale linearly, so the ratio holds: **there is no crossover to wait for.**

Cold build is the `std` BMI, once — **5.1 s** on GCC, **4.6 s** on Clang. That is the whole "modules are slower" story.

`TU_COUNT=64 ./scripts/time-builds.sh g++`. Check what your `g++` is first: on Fedora it is a ccache shim, and a cache hit rebuilds 64 header TUs in 0.07 s.



Week-one crash: include after import

Even with `import std;` as the default, this is the bug you will hit before lunch — the moment a leftover header or a third-party include shows up.

```
import tally;           // or import std;  
#include <iostream>      // GCC: redefinition of std::__is_constant_evaluated
```

Rule: do not `#include` a standard header *after* an `import` that already brought those declarations in. Prefer:

```
import std;  
import tally;
```

If you still have to include something, include it **first**. Example 02 ships `include-err-after-import.cpp` (not in the default build) for the live diagnostic.



Macros do not cross import

```
export module tally.log;  
#define TALLY_TRACE(msg) ::tally::trace_impl(__FILE__, __LINE__, (msg))  
export void trace(std::string_view);
```

```
import tally.log;  
TALLY_TRACE("hi");
```

Clang:

```
error: use of undeclared identifier 'TALLY_TRACE'
```

GCC:

```
error: 'TALLY_TRACE' was not declared in this scope
```

That is not a compiler bug. **Macros are not declarations.** `export` exports declarations.

And as of C++26 you cannot even write the module *name* with a macro.



What to do with macros

Want	Do
Logging / assert	Export a function . Keep the macro inside the module unit.
Feature-test	<code>__cpp_lib_modules</code> , your own <code>inline constexpr</code> flags
Header-only config (<code>JSONLITE_MAX_DEPTH</code>)	<code>#define</code> it in the GMF , then <code>#include</code>
A macro clients must type	You still need a header . Modules do not replace every header.

`#include "tally/log.hpp"` pastes `TALLY_TRACE` into the consumer.
`import tally.log;` does not. Example 03 builds both, side by side.



Templates

Nothing about templates changed except **reachability**.

```
export module tally;

export template<class Rep>
class Money { /* all members here */ };

export template<class Rep>
std::string format(Money<Rep> m) {      // body in the interface
    return std::to_string(m.amount());
}

template<class T>                        // not exported
constexpr T mix(T a, T b) { return a ^ b; }
```

Importers can instantiate **Money** and **format**.

They cannot name **mix**. Clang:

```
error: declaration of 'mix' must be imported from module 'tally'
      before it is required
note: declaration here is not visible
```



Template rules that matter on Monday

1. A function template the importer must instantiate needs its **definition reachable** from the interface BMI. Put it in the PMIU or an interface partition.
2. A non-template member can live in `module tally;` as before.
3. Explicit instantiations in an implementation unit still work — for the specializations you list, not for the ones you forgot.
4. `export` is not `inline`. An exported non-inline function is emitted **once**, in the module's object file. That is a binary-size win over header-only `inline`.
5. **Testing.** Public API: an ordinary TU — `#include <catch2/...>` **first**, then `import tally;`. No mainstream test framework ships a module interface yet.
Non-exported entities: the test must *be* a module unit — that is what `extern "C++" int main()` is for.



Header-only dependencies

nlohmann/json, fmt, your internal `expected.hpp` : no module, lots of macros, lots of templates.

```
module;
#define JSONLITE_MAX_DEPTH 8    // configure *before* the include
#include "jsonlite.hpp"
export module jsonwrap;

export namespace jsonwrap {
    using jsonlite::Value;      // attached to the global module
    Value parse_config(std::string_view);
}
```

`JSONLITE_MAX_DEPTH` does not leak to importers.

`export using` does not steal jsonlite's ABI — the type stays in the global module.

That is the wrapping pattern. Example 04 (`jsonwrap.cppm`).



Header units — do not wait for them

The standard lets you write `import <string>;`.

GCC 16 builds one. `--compile-std-module` produces a `<bits/stdc++.h>` header unit — and once it exists, GCC rewrites `#include <vector>` into an import of it. Silently, in your build.

CMake still builds none.

Daniel Ruoso's C++Now 2023 talk is still the right explanation of why they are hard.

Until they exist in the build system, the portable options are:

1. `import std;` for the standard library (C++23 / 26 — this talk's default)
2. GMF `#include` for third-party headers that still use macros as config
3. A named-module wrapper around the header-only library (example 04)

Do not design the migration around header units.



Mixed codebase — you will live here for years

```
apps/legacy-app.cpp    #include "tally/account.hpp"  
apps/module-app.cpp    import tally;  
apps/dual-app.cpp      #include "tally/tally.hpp"    // picks a side
```

```
module;  
#include "tally/account.hpp"  
export module tally;  
export namespace tally { using ::tally::Account; using ::tally::Money; }
```

- `export using` / `extern "C++"`: names stay on the **global module** — header TU and module TU link
- Include in the purview **changes mangling**. Don't mix leftover `#include` of the same types
- `nm -C libtally.a` · `nm -DC libtally.so`: global `Account::credit` vs module `Account@tally::credit` (same address = alias)
- Three visibilities, all must agree: `export` (name it) · ELF (`-fvisibility` + attribute) · version script (cannot un-hide). `__ZGIW*` is a real symbol



Dual-mode umbrella

```
// include/tally/tally.hpp
#pragma once
#ifdef TALLY_USE_MODULE
    import tally;
#else
    #include "tally/account.hpp"
    #include "tally/money.hpp"
#endif
```

Convert **translation units**, not the whole repo.

- New files: `import tally;`
- Old files: keep including
- A TU that needs both: umbrella + `TALLY_USE_MODULE`

Do not `#include` a modularized header **after** `import tally` of the same entities.
Redeclarations are where both compile time and compiler bugs hide.



examples/06-trading — a library you can ship

```
libtrading    STATIC .a (no PIC)    or    SHARED .so (-fPIC)
  templates:  Ticks<>    Blotter::fee_ticks<>    (in headers)
  sources:    order.cpp book.cpp blotter.cpp    (scan OFF)
```

```
oms_legacy    #include "trading/trading.hpp"
oms_modules    #define TRADING_USE_MODULE    then the umbrella
post_trade    import std; import trading;
```

Same session on the two OMS binaries. `export using` keeps names on the **global module**.

```
./nm-symbols.sh → trading::Order::fill, not Order@trading.
```



What goes in the tarball

BMIs are **not** a distribution format. Consumers rebuild them from the `.cppm`.

```
target_compile_features(trading PUBLIC cxx_std_26)    # exported module target
install(TARGETS trading EXPORT tradingTargets
  FILE_SET HEADERS      DESTINATION include
  FILE_SET CXX_MODULES  DESTINATION lib/cmake/trading/src)
install(EXPORT tradingTargets NAMESPACE trading::
  DESTINATION lib/cmake/trading CXX_MODULES_DIRECTORY cxx-modules) # + config file
```

Path	What
<code>include/trading/*.hpp</code>	templates + umbrella — every header the GMF <code>#include S</code>
<code>lib/cmake/trading/</code>	<code>tradingConfig.cmake</code> · <code>cxx-modules/</code> · <code>src/*.cppm</code>
<code>lib/libtrading.{a,so}</code>	objects; PIC only on the <code>.so</code>
<code>bin/oms_*</code> <code>post_trade</code>	the delivery

`./package.sh` → the two tarballs. Never ship `.pcm` / `.gcm`.

No `install(EXPORT ...)` → no `find_package`. That is the next slide.



What the other project types

A separate project. It shares no code with `06-trading` — only the install prefix.

```
find_package(trading 1.0 CONFIG REQUIRED)    # found via CMAKE_PREFIX_PATH
target_link_libraries(app PRIVATE trading::trading)
```

```
import std;
import trading;
```

`find_package` + `target_link_libraries`. No module-specific consumer step.

They rebuild your `.cppm` into **their** BMI, with **their** flags.

That is why BMIs do not ship — and why `cxx_std_26` is `PUBLIC` on `trading`.

The import-std gate does not travel — the consumer sets its own.



Incremental adoption

Week 1. One leaf library. One PMIU. C++26, minimum C++23. `import std;` . CMake 3.30, Ninja, `FILE_SET` . UUID + standard + `CMAKE_CXX_EXTENSIONS OFF` before `project()` . Measure the build.

Week 2. Implementation units for the `.cpp` files. Turn scanning **off** (`CMAKE_CXX_SCAN_FOR_MODULES=0`) on targets that still have no `import` .

Month 1. Interface partitions only if the PMIU is unwieldy. Wrapper module for the worst header-only dependency. Dual-mode umbrella for consumers.

Never start by modularizing Boost. Start with *your* leaf. The GMF remains for third-party headers that are not modules.



What still hurts — say this to your manager

- **GCC 16** still needs `-fmodules` and has no private module fragment
- `import std;` is experimental in CMake; the UUID is a compatibility break by design
- **BMIs are not portable** across compiler, flags, or stdlib — and since Clang 22 they are *reduced* by default, deliberately incomplete. You ship sources, not `.pcm`
- **Scanning** every C++20 file is real cost; disable it on non-module targets
- **Include-after-import** is a class of bugs, not a one-off
- **Macros** that are part of your public API still need a header

Modules stopped being experimental around 2025.

The *ecosystem* is usable. It is not boring yet.



Recap

```
export module tally;           // one name clients import
export import :money;          // partitions are internal
module tally;                  // implementation unit, ordinary .cpp
```

```
import std;                    // C++23 / 26 – default in this talk
import tally;                  // one import per line; includes first
```

```
set(CMAKE_EXPERIMENTAL_CXX_IMPORT_STD "...") # before project()
set(CMAKE_CXX_STANDARD 26) set(CMAKE_CXX_STANDARD_REQUIRED ON) set(CMAKE_CXX_EXTENSIONS OFF)
set(CMAKE_CXX_MODULE_STD ON)
project(tally LANGUAGES CXX)
# FILE_SET CXX_MODULES for the PMIU; ordinary .cpp for impl units
```

- App: one `import` per line; includes first; link `.o` not BMIs
- Library: one PMIU; never `export import std;`; scan off on non-module TUs
- Ship: `.cppm` + every GMF header; never `.pcm` / `.gcm`
- Macros do not `export`. Template bodies live in the interface. Convert TUs, not trees. `nm -C : Account::credit` VS `@tally`



Where to look it up on the plane home

- github.com/erez-strauss/cxx-modules — talk materials
- [Modules \(C++\)](#) — Wikipedia
- Layout: `examples/00-simple-example ... 06-trading`
- Build: `./scripts/build-all.sh g++` (tally 01-05); `clang = clang++`
- CMake freeze: `cmake/EnableImportStd.cmake` — UUID + standard + extensions **before** `project()`
- Recommended toolchain: CMake 4.4+ · Ninja 1.11 · GCC 16.1+ · Clang 22
- [cmake-cxxmodules\(7\)](#)
- [Clang: Standard C++ Modules](#)
- [cppreference: Modules \(since C++20\)](#)
- [Chuanqi Xu, 2025: Practical Insights](#)
- [Chuanqi Xu, 2025: Best Practices](#)
- [Daniel Ruoso, C++Now 2023: header units](#)
- [Tyler Drake, CppCon 2025 lightning](#)
- [P3034R1](#) — module names are not macros
- [P3618R0](#) — `extern "C++" int main()` in an implementation unit

